

React + Tailwind CSS Internship Assignment

One-Week Project: Job Finder Dashboard

Assignment Objective

Build a clean, responsive **Job Finder web application** using React.js and Tailwind CSS. The purpose of this assignment is to demonstrate your understanding of React fundamentals, component-based development, routing, state management, reusable UI, filtering, and browser storage. **No backend is required.** All job information should come from dummy data.

1. Technology Requirements

- React.js with Vite
- Tailwind CSS
- React Router DOM
- JavaScript (ES6+)
- Git & GitHub
- LocalStorage

2. Project Requirements

The application should allow a user to browse available jobs, search and filter jobs, view complete job details, save jobs, and view their saved jobs. The interface must be responsive and built using reusable React components.

3. Required Pages

A. Home Page

- Responsive Navbar with navigation links.
- Hero section with a clear heading and job-search call to action.
- Search bar.
- Featured Jobs section displaying 3–4 jobs.
- A short 'Why use our platform?' or features section.
- Footer.

B. Jobs Page

- Display 8–10 jobs using dummy data.
- Create each job using a reusable JobCard component.
- Each card must display: Job Title, Company, Location, Job Type, Salary, Skills, Apply/View Details button, and Save button.
- Implement search by job title, company, or skill.
- Implement filtering by job type.
- Implement filtering by location.
- Allow users to save and unsave jobs.

C. Job Details Page

- Use a dynamic route such as `/jobs/:id`.
- Display Job Title, Company, Location, Salary, Job Type, Description, Requirements, Skills, and an Apply button.
- The page must display the correct job based on the selected job ID.

D. Saved Jobs Page

- Display all jobs saved by the user.
- Allow a user to remove a saved job.
- Use `LocalStorage` so saved jobs remain available after refreshing the browser.
- Show a suitable empty state when there are no saved jobs.

4. React Requirements

- Use functional React components.
- Use props to pass data to reusable components.
- Use `useState` for interactive state.
- Use `useEffect` where appropriate.
- Use `.map()` to render job data dynamically.
- Use conditional rendering for empty states and UI changes.
- Use React Router DOM for navigation.
- Use a dynamic route for Job Details.
- Use event handling for search, filters, saving, and other interactions.
- Use `LocalStorage` for saved jobs.
- Avoid writing the entire application inside one component.

5. Required Reusable Components

- Navbar
- Footer
- JobCard
- SearchBar
- Filter
- Button

The JobCard component must be reusable. Job information should be passed through props rather than hard-coded into separate cards.

6. Tailwind CSS Requirements

- Use Tailwind CSS for the main styling.
- Create a clean and modern interface.
- Use consistent spacing, typography, buttons, cards, and form controls.
- Include hover, focus, and active states where appropriate.
- The application must be responsive on mobile, tablet, and desktop.
- Pay attention to layout changes at different screen sizes.

7. Dummy Job Data

Create a local data file such as `src/data/jobs.js`. Each job should contain at least:

- `id`
- `title`
- `company`
- `location`
- `jobType`

- salary
- skills
- description
- requirements

8. Suggested Folder Structure

```
src/  
  components/  
    ■ ■■■■ Navbar.jsx  
    ■ ■■■■ Footer.jsx  
    ■ ■■■■ JobCard.jsx  
    ■ ■■■■ SearchBar.jsx  
    ■ ■■■■ Filter.jsx  
  ■  
  pages/  
    ■ ■■■■ Home.jsx  
    ■ ■■■■ Jobs.jsx  
    ■ ■■■■ JobDetails.jsx  
    ■ ■■■■ SavedJobs.jsx  
  ■  
  data/  
    ■ ■■■■ jobs.js  
  ■  
  ■■■■ App.jsx  
  ■■■■ main.jsx
```

9. Functional Requirements

- Navigation links must work correctly.
- Clicking a job must open its corresponding details page.
- Search results must update according to the user's search input.
- Job type and location filters must work correctly.
- Save/unsave must update the UI immediately.
- Saved jobs must persist after browser refresh using LocalStorage.
- The application must handle cases where no jobs match the search/filter.
- Buttons and interactive elements should provide appropriate feedback.

10. One-Week Development Plan

Day	Task
Day 1	Project setup, Tailwind configuration, GitHub repository, folder structure, Navbar and Footer.
Day 2	Home Page and Jobs Page, dummy job data, reusable JobCard.
Day 3	React Router, dynamic Job Details page, navigation.
Day 4	Search functionality and job type/location filtering.
Day 5	Save/unsave jobs and LocalStorage, Saved Jobs page.
Day 6	Responsive design, empty states, UI polish, bug fixing.
Day 7	Testing, README, GitHub cleanup, final submission and demo.

11. Submission Requirements

- Submit the complete project through a GitHub repository.
- Repository must contain the source code and package.json.
- Do not upload node_modules.
- The project must run successfully after npm install and npm run dev.
- Include a README.md with project description, features, setup instructions, and screenshots.
- Submit the GitHub repository link in Google Classroom.
- A live deployment link is optional but recommended.
- Be prepared to give a short demonstration of the project.

12. Code Quality Expectations

- Use meaningful names for components, variables, and functions.
- Keep components small and focused.
- Avoid unnecessary repetition.
- Keep dummy data separate from UI components.
- Maintain a clean folder structure.
- Remove unused imports, variables, and console errors before submission.
- Use Git commits throughout development instead of submitting one final commit only.

13. Evaluation Criteria — 100 Marks

Criteria	Marks
React concepts & componentization	20
Tailwind CSS & responsive design	20
Routing & dynamic pages	15
Search & filtering	15
LocalStorage / saved jobs	10
UI/UX & visual presentation	10
Code quality & GitHub practices	10
Total	100

14. Optional Bonus Features

- Dark mode.
- Toast notifications.
- Pagination.
- Debounced search.
- Skeleton loading cards.
- Animations and transitions.
- Job category filtering.
- A polished mobile navigation menu.

15. Important Instructions

- This is a frontend assignment. Do not create a backend.
- Do not use MongoDB, Express.js, or Node.js APIs for this task.
- Use dummy/local data.
- The main focus is understanding React and Tailwind CSS, not simply creating a large number of pages.
- You should be able to explain every major part of your submitted code during the final review.
- Copied projects or code that the intern cannot explain may not receive full marks.

Final Goal: By the end of one week, you should have a polished, responsive Job Finder Dashboard that demonstrates practical React development, reusable components, routing, state management, search/filtering, LocalStorage, and Tailwind CSS.